A Lab Project Report on

TEMPERATURE CONVERTER USING USER DEFINED FUNCTIONS IN FLUTTER

Submitted in partial fulfilment of the requirement of the lab

UDF FLUTTER LAB

III-B. TECH. I SEM

in

Department of CSE(AI & ML)

Submitted by

A.ANIRUDH (247R1A6603)

D.SRIKAR REDDY (247R1A6621)

G.PRASHANTH (247R1A6622)



DEPARTMENT OF CSE(AI & ML)

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NBA, Approved by AICTE, Affiliated to JNTUH)

Kandlakoya, Medchal Road, Hyderabad-501401

2026-2027

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Approved by AICTE, Affiliated to JNTUH, Kukatpally, Hyderabad)

Kandlakoya, Medchal Road, Hyderabad-501401.

DEPARTMENT OF CSE(AI & ML)



CERTIFICATE

This is to certify that a Mini Project entitled with: “TEMPERATURE CONVERTER USING USER DEFINED FUNCTIONS IN FLUTTER” is being Submitted by A.ANIRUDH (247R1A6603), D.SRIKAR REDDY (247R1A6621), G.PRASHANTH (247R1A6622) in partial fulfilment of the requirement for completion of the UDF FLUTTER LAB III B.Tech I- Semester is a record of a bonafide work carried out under guidance and supervision.

Signature of Faculty **Signature of the HOD**

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. A. Raji Reddy, Director, and Dr. S Rao Chintalapudi, Head of Department,** Department of CSE (AI & ML), CMR TECHNICAL CAMPUS, for their inspiration and valuable guidance during the entire duration. We are extremely thankful to our Assistant Professor **G M Subhani**, Department of CSE (AI & ML), for his constant guidance, encouragement, and moral support throughout the project. We express our gratitude to all staff members and friends for the help and coordination they extended in bringing this mini project to a successful completion on time.

A.ANIRUDH (247R1A6603)
D.SRIKAR REDDY (247R1A6621)
G.PRASHANTH (247R1A6622)

The table of contents outlines the project's structure, covering:

- Abstract
- CHAPTER 1: Introduction
    - Introduction to the Temperature Converter (Definition, purpose, importance, features)
    - Introduction to Flutter and Dart (History, features, why Flutter is used for the project, advantages)
- CHAPTER 2: Literature Survey
    - Temperature Conversion Applications (Overview, features/limitations, comparison)
    - Graphical User Interface (GUI) for the Converter (Importance, examples)
- CHAPTER 3: Analysis and Design
    - System Architecture (Overview, components/modules)
    - System Design (Detailed design)
- CHAPTER 4: Implementation
    - User Defined Functions in Dart (Introduction, use in the converter)
    - Code (Complete code for the project)
- CHAPTER 5: Results
    - Output of the Temperature Converter (Examples, discussion of results)
- CHAPTER 6: Conclusion and future scope
    - Summary of the project (Recap, limitations)
    - Future scope and enhancements (Ideas for improvements)
- References

Abstract

This lab project presents a Temperature Converter application built with Flutter and Dart to demonstrate the use of User Defined Functions (UDFs). The app converts values between the Celsius, Fahrenheit and Kelvin scales, rejects temperatures below absolute zero, shows the formula used, and presents the result in an animated card with light and dark Material 3 themes. All calculations are written as pure Dart functions in a separate utility file, so the user interface only collects input and calls the functions. This separation of concerns makes the logic reusable and easy to test, and it is checked by 12 unit tests and 5 widget tests. The project shows how modular functions, input validation and a clean widget structure lead to a maintainable mobile application that can run on Android, iOS, web and desktop from one code base.

CHAPTER 1: Introduction

Introduction to the Temperature Converter

1.1 Definition and purpose of a temperature converter: A temperature converter is an application that changes a temperature value from one scale to another, for example from Celsius to Fahrenheit. The purpose of this project is to build such a converter in Flutter while demonstrating User Defined Functions, which are functions written by the programmer for a specific task instead of being provided by the language.

1.2 Importance of temperature conversion in daily life: Temperature is reported in different scales in different countries and fields. Weather reports, cooking, medicine and laboratory work use Celsius, Fahrenheit or Kelvin. A reliable converter helps users read values correctly, and the underlying formulas are a simple but complete example for learning functions and validation.

1.3 Features of the Temperature Converter: The app supports all six conversions between Celsius, Fahrenheit and Kelvin, a swap button that exchanges the units and recalculates, and a clear button that resets the screen. It validates empty, non-numeric and below-absolute-zero input, displays the formula used, shows the result in an animated card, adapts to light and dark mode and provides an information dialog that lists the UDFs.

Introduction to Flutter and Dart

1.4 Brief history and features of Flutter and Dart: Dart is a strongly typed, object-oriented language developed by Google and announced in 2011. Flutter is Google's open source UI toolkit, first announced in 2015 and released as version 1.0 in December 2018. It builds the interface from widgets, supports hot reload and compiles to native code for Android, iOS, web, Windows, macOS and Linux from a single code base.

1.5 Why Flutter and Dart are used for the project: Dart supports top-level functions with explicit parameter and return types, sound null safety and the flutter_test package for automated testing, which suit a project about UDFs. Flutter provides ready-made Material 3 widgets for text input, dropdowns, buttons and cards, so effort can be spent on the logic and structure rather than on drawing the interface.

1.6 Advantages of using Flutter for the converter: A single code base runs on many platforms, hot reload makes interface changes visible in seconds, and the widget tree encourages small reusable components such as the unit dropdown. The built-in test

framework supports both unit tests for the functions and widget tests that simulate taps and typing without an emulator.

CHAPTER 2: Literature Survey

Temperature Conversion Applications

2.1 Overview of existing conversion applications: Temperature conversion is available in phone unit-converter apps, in web search results, in scientific calculators and in many tutorial projects. Most of them convert between Celsius and Fahrenheit and some add Kelvin or other scales such as Rankine.

2.2 Features and limitations of existing applications: Many beginner examples place the formula directly inside a button callback, which mixes calculation with the interface and makes the logic hard to reuse or test. Few of them check whether the entered value is physically possible, and error handling for empty or non-numeric text is often missing. Some commercial apps are limited to one platform or show advertisements.

2.3 Comparison of existing applications with the proposed application: Table 2.1 compares typical approaches with this project.

Table 2.1: Comparison of temperature converters

Feature	Typical converter apps	Basic tutorial projects	This project
Logic separated from the UI	Hidden from the user	Usually inside onPressed	Yes, pure UDFs in lib/utils
Absolute zero validation	Rare	No	Yes, for all three scales
Input error handling	Varies	Often missing	Empty, non-numeric and physical checks
Automated tests	Not visible	Few or none	12 unit tests and 5 widget tests
Light and dark theme	Varies	Default theme	Material 3, follows the system
Platforms	Usually one	Usually one	Android, iOS, web and desktop

Graphical User Interface (GUI) for the Converter

2.4 Importance of a GUI in a converter: A clear interface lets the user enter a value, choose units and read the result without knowing the formulas. Good feedback, such as an inline error message under the text field, prevents mistakes, and a large result card makes the answer easy to read on a small screen.

2.5 Examples of GUI toolkits and widgets used: Mobile interfaces are commonly built with Android Views, SwiftUI or Flutter. Flutter offers two widget libraries, Material and Cupertino. This project uses Material 3 widgets: TextField for input, DropdownButton for the units, FilledButton and OutlinedButton for actions, Card for the result, SnackBar for alerts and AlertDialog for the UDF information dialog.

CHAPTER 3: Analysis and Design

System Architecture

3.1 Overview of the system architecture: The application is divided into a presentation layer and a business logic layer, with a test layer that checks both, as shown in Figure 3.1. The presentation layer contains the screen and the reusable dropdown widget. The business logic layer contains only pure Dart functions and constants and has no Flutter code, so it can be tested without the Flutter engine.

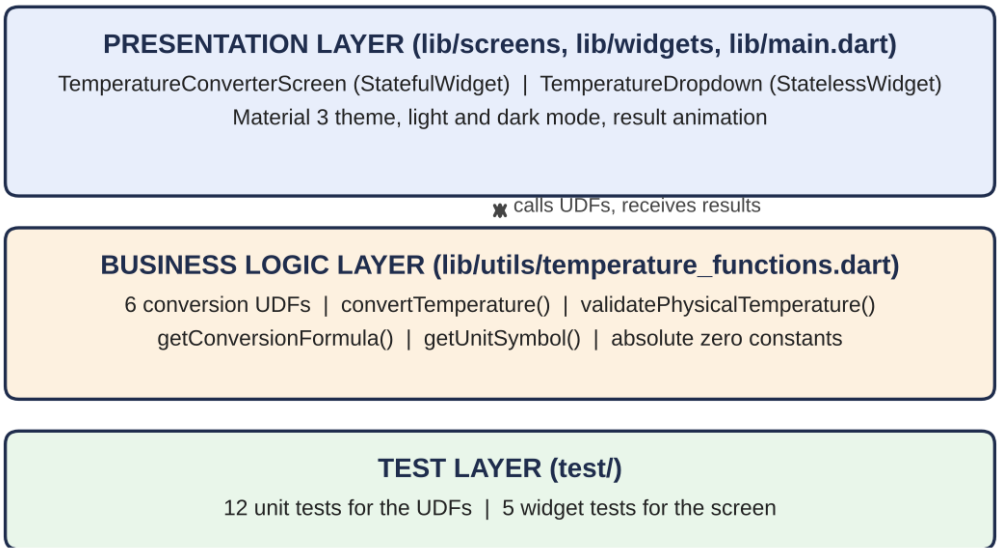


Figure 3.1: Layers of the Temperature Converter

3.2 Components and modules used in the system: main.dart starts the app and defines the light and dark Material 3 themes. screens/temperature_converter_screen.dart holds the stateful screen with the input field, unit selectors, buttons, result card and animation. widgets/temperature_dropdown.dart is the reusable unit selector used for both the From and To fields. utils/temperature_functions.dart contains the UDFs. The test folder contains temperature_functions_test.dart and widget_test.dart.

System Design

3.3 Detailed design of the system: The converter implements the six formulas in Table 3.1. When the user presses Convert, the screen hides the keyboard, checks for empty input, parses the text with double.tryParse, calls validatePhysicalTemperature to reject values below absolute zero (0 K, -273.15 °C and -459.67 °F), calls convertTemperature, and finally updates the state and plays a scale and fade animation. If any check fails, an error message is shown and the result card is hidden. The swap button exchanges the units and repeats the conversion if a result is already shown. The clear button empties the field, restores Celsius to Fahrenheit and resets the animation.

Table 3.1: Conversion formulas and the functions that implement them

Conversion	Formula	UDF
Celsius to Fahrenheit	F = (C x 9/5) + 32	celsiusToFahrenheit
Fahrenheit to Celsius	C = (F - 32) x 5/9	fahrenheitToCelsius
Celsius to Kelvin	K = C + 273.15	celsiusToKelvin
Kelvin to Celsius	C = K - 273.15	kelvinToCelsius
Fahrenheit to Kelvin	K = (F - 32) x 5/9 + 273.15	fahrenheitToKelvin
Kelvin to Fahrenheit	F = (K - 273.15) x 9/5 + 32	kelvinToFahrenheit

CHAPTER 4: Implementation

User Defined Functions in Dart

4.1 Introduction to User Defined Functions in Dart: A function is a named block of code that performs one task. A User Defined Function is written by the programmer, as opposed to built-in functions such as print() or double.parse(). A Dart function has a return type, a name, parameters and a body, for example double celsiusToFahrenheit(double celsius). Functions that always return the same output for the same input and change nothing outside themselves are called pure functions. UDFs remove repeated code, divide a program into small units, make testing easy and localise future changes.

4.2 Use of User Defined Functions in the converter: The project defines ten UDFs in lib/utils/temperature_functions.dart: six conversion functions, the dispatcher convertTemperature, the checker validatePhysicalTemperature, and the helpers getConversionFormula and getUnitSymbol. All conversion functions are pure. The screen never contains a formula; it only calls these functions, which is why the same logic can be reused and tested on its own.

Code

4.3 Complete code for the Temperature Converter project: The complete source code of the application and its tests is given below.

Listing 4.1: lib/main.dart

```
import 'package:flutter/material.dart';

import 'screens/temperature_converter_screen.dart';

void main() {
  runApp(const TemperatureConverterApp());
}

class TemperatureConverterApp extends StatelessWidget {
  const TemperatureConverterApp({super.key});

  ThemeData _buildTheme(Color seed, Brightness brightness) {
    return ThemeData(
      useMaterial3: true,
      colorScheme: ColorScheme.fromSeed(seedColor: seed, brightness: brightness),
      fontFamily: 'Roboto',
      appBarTheme: const AppBarTheme(centerTitle: true, elevation: 0),
      inputDecorationTheme: InputDecorationTheme(
        border: OutlineInputBorder(borderRadius: BorderRadius.circular(16)),
        filled: true,
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Temperature Converter',
      debugShowCheckedModeBanner: false,
      theme: _buildTheme(const Color(0xFF1E40AF), Brightness.light), // Royal Indigo
```

```
      darkTheme: _buildTheme(const Color(0xFF3B82F6), Brightness.dark),
      themeMode: ThemeMode.system,
      home: const TemperatureConverterScreen(),
    );
  }
}
```

Listing 4.2: lib/utils/temperature_functions.dart (the UDFs)

```
// Pure Dart User Defined Functions (UDFs) for temperature conversion.
// Contains no Flutter UI code.

const double absoluteZeroKelvin = 0.0;
const double absoluteZeroCelsius = -273.15;
const double absoluteZeroFahrenheit = -459.67;

const String unitCelsius = 'Celsius (°C)';
const String unitFahrenheit = 'Fahrenheit (°F)';
const String unitKelvin = 'Kelvin (K)';

const List<String> supportedUnits = [
  unitCelsius,
  unitFahrenheit,
  unitKelvin,
];

// 1. Celsius to Fahrenheit
double celsiusToFahrenheit(double celsius) {
  return (celsius * 9 / 5) + 32;
}

// 2. Fahrenheit to Celsius
double fahrenheitToCelsius(double fahrenheit) {
  return (fahrenheit - 32) * 5 / 9;
}

// 3. Celsius to Kelvin
double celsiusToKelvin(double celsius) {
  return celsius + 273.15;
}

// 4. Kelvin to Celsius
double kelvinToCelsius(double kelvin) {
  return kelvin - 273.15;
}

// 5. Fahrenheit to Kelvin
double fahrenheitToKelvin(double fahrenheit) {
  return (fahrenheit - 32) * 5 / 9 + 273.15;
}

// 6. Kelvin to Fahrenheit
double kelvinToFahrenheit(double kelvin) {
  return (kelvin - 273.15) * 9 / 5 + 32;
}

/// Dispatcher UDF: routes a conversion request to the matching UDF.
double convertTemperature(double value, String fromUnit, String toUnit) {
  if (fromUnit == toUnit) return value;

  if (fromUnit == unitCelsius && toUnit == unitFahrenheit) return celsiusToFahrenheit(value);
  if (fromUnit == unitCelsius && toUnit == unitKelvin) return celsiusToKelvin(value);

  if (fromUnit == unitFahrenheit && toUnit == unitCelsius) return fahrenheitToCelsius(value);
  if (fromUnit == unitFahrenheit && toUnit == unitKelvin) return fahrenheitToKelvin(value);

  if (fromUnit == unitKelvin && toUnit == unitCelsius) return kelvinToCelsius(value);
  if (fromUnit == unitKelvin && toUnit == unitFahrenheit) return kelvinToFahrenheit(value);

  return value; // Fallback safety
}

/// Returns an error message if [value] is below absolute zero, else null.
String? validatePhysicalTemperature(double value, String unit) {
  if (unit == unitKelvin && value < absoluteZeroKelvin) {
    return 'Invalid temperature: Kelvin cannot be below 0 K (Absolute Zero).';
  }
  if (unit == unitCelsius && value < absoluteZeroCelsius) {
    return 'Invalid temperature: Celsius cannot be below -273.15 °C (Absolute Zero).';
  }
  if (unit == unitFahrenheit && value < absoluteZeroFahrenheit) {
    return 'Invalid temperature: Fahrenheit cannot be below -459.67 °F (Absolute Zero).';
  }
  return null; // Valid
}

String getConversionFormula(String fromUnit, String toUnit) {
  if (fromUnit == toUnit) return 'Identical units selected: No conversion needed (Result = Input).';
  if (fromUnit == unitCelsius && toUnit == unitFahrenheit) return '°F = (°C × 9/5) + 32';
  if (fromUnit == unitFahrenheit && toUnit == unitCelsius) return '°C = (°F - 32) × 5/9';
  if (fromUnit == unitCelsius && toUnit == unitKelvin) return 'K = °C + 273.15';
  if (fromUnit == unitKelvin && toUnit == unitCelsius) return '°C = K - 273.15';
  if (fromUnit == unitFahrenheit && toUnit == unitKelvin) return 'K = (°F - 32) × 5/9 + 273.15';
```

```
  if (fromUnit == unitKelvin && toUnit == unitFahrenheit) return '°F = (K - 273.15) × 9/5 + 32';
  return 'Formula not available';
}

String getUnitSymbol(String unit) {
  if (unit == unitCelsius) return '°C';
  if (unit == unitFahrenheit) return '°F';
  if (unit == unitKelvin) return 'K';
  return '';
}
```

Listing 4.3: lib/widgets/temperature_dropdown.dart

```
import 'package:flutter/material.dart';

import '../utils/temperature_functions.dart';

/// Reusable unit selector used for both the "From" and "To" pickers.
class TemperatureDropdown extends StatelessWidget {
  final String label;
  final String value;
  final ValueChanged<String?> onChanged;

  const TemperatureDropdown({
    super.key,
    required this.label,
    required this.value,
    required this.onChanged,
  });

  IconData _iconFor(String unit) {
    if (unit == unitCelsius) return Icons.ac_unit_rounded;
    if (unit == unitFahrenheit) return Icons.local_fire_department_rounded;
    return Icons.science_rounded;
  }

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;

    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          label,
          style: Theme.of(context).textTheme.labelLarge?.copyWith(
                color: colorScheme.onSurfaceVariant,
              ),
        ),
        const SizedBox(height: 6),
        Container(
          padding: const EdgeInsets.symmetric(horizontal: 12),
          decoration: BoxDecoration(
            color: colorScheme.surfaceContainerHighest,
            borderRadius: BorderRadius.circular(16),
            border: Border.all(color: colorScheme.outlineVariant),
          ),
          child: DropdownButtonHideUnderline(
            child: DropdownButton<String>(
              value: value,
              isExpanded: true,
              borderRadius: BorderRadius.circular(16),
              onChanged: onChanged,
              items: supportedUnits
                  .map(
                    (unit) => DropdownMenuItem<String>(
                      value: unit,
                      child: Row(
                        children: [
                          Icon(_iconFor(unit), size: 20, color: colorScheme.primary),
                          const SizedBox(width: 8),
                          Flexible(
                            child: Text(unit, overflow: TextOverflow.ellipsis),
                          ),
                        ],
                      ),
                    ),
                  )
                  .toList(),
            ),
          ),
        ),
      ],
    );
  }
}
```

Listing 4.4: lib/screens/temperature_converter_screen.dart

```
import 'package:flutter/material.dart';

import '../utils/temperature_functions.dart';
import '../widgets/temperature_dropdown.dart';
```

```
class TemperatureConverterScreen extends StatefulWidget {
  const TemperatureConverterScreen({super.key});

  @override
  State<TemperatureConverterScreen> createState() => _TemperatureConverterScreenState();
}

class _TemperatureConverterScreenState extends State<TemperatureConverterScreen>
    with SingleTickerProviderStateMixin {
  final TextEditingController _inputController = TextEditingController();

  String _fromUnit = unitCelsius;
  String _toUnit = unitFahrenheit;

  double? _inputTemperature;
  double? _convertedResult;
  String? _errorMessage;
  bool _hasConverted = false;

  late AnimationController _resultAnimController;
  late Animation<double> _scaleAnimation;
  late Animation<double> _fadeAnimation;

  @override
  void initState() {
    super.initState();
    _resultAnimController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 400),
    );
    _scaleAnimation = CurvedAnimation(parent: _resultAnimController, curve: Curves.easeOutBack);
    _fadeAnimation = CurvedAnimation(parent: _resultAnimController, curve: Curves.easeIn);
  }

  @override
  void dispose() {
    _inputController.dispose();
    _resultAnimController.dispose();
    super.dispose();
  }

  void _showError(String message, {bool snackBar = false}) {
    setState(() {
      _errorMessage = message;
      _convertedResult = null;
      _hasConverted = false;
    });
    if (snackBar) {
      ScaffoldMessenger.of(context)
        ..hideCurrentSnackBar()
        ..showSnackBar(
          SnackBar(
            content: Text(message),
            behavior: SnackBarBehavior.floating,
            backgroundColor: Theme.of(context).colorScheme.error,
          ),
        );
    }
  }

  void _performConversion() {
    FocusScope.of(context).unfocus();

    final rawInput = _inputController.text.trim();
    if (rawInput.isEmpty) {
      _showError('Please enter a temperature value.', snackBar: true);
      return;
    }

    final value = double.tryParse(rawInput);
    if (value == null) {
      _showError('Please enter a valid numeric value (e.g. 25 or 98.6).');
      return;
    }

    final physicalError = validatePhysicalTemperature(value, _fromUnit);
    if (physicalError != null) {
      _showError(physicalError);
      return;
    }

    setState(() {
      _inputTemperature = value;
      _convertedResult = convertTemperature(value, _fromUnit, _toUnit);
      _errorMessage = null;
      _hasConverted = true;
    });
    _resultAnimController.forward(from: 0.0);
  }

  void _swapUnits() {
    setState(() {
      final temp = _fromUnit;
```

```
    _fromUnit = _toUnit;
    _toUnit = temp;
  });
  if (_hasConverted) _performConversion();
}

void _clearAll() {
  _inputController.clear();
  _resultAnimController.reset();
  setState(() {
    _fromUnit = unitCelsius;
    _toUnit = unitFahrenheit;
    _inputTemperature = null;
    _convertedResult = null;
    _errorMessage = null;
    _hasConverted = false;
  });
}

void _showUdfInfoDialog(BuildContext context) {
  showDialog<void>(
    context: context,
    builder: (context) => AlertDialog(
      icon: const Icon(Icons.functions_rounded),
      title: const Text('About the UDFs'),
      content: const SingleChildScrollView(
        child: Text(
          'All conversion logic lives in lib/utils/temperature_functions.dart as '
          'pure User Defined Functions:\n\n'
          '• celsiusToFahrenheit()\n'
          '• fahrenheitToCelsius()\n'
          '• celsiusToKelvin()\n'
          '• kelvinToCelsius()\n'
          '• fahrenheitToKelvin()\n'
          '• kelvinToFahrenheit()\n'
          '• convertTemperature() - dispatcher\n'
          '• validatePhysicalTemperature() - absolute zero check\n'
          '• getConversionFormula() / getUnitSymbol() - display helpers\n\n'
          'The UI only collects input and calls these functions.',
        ),
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('Got it'),
        ),
      ],
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Temperature Converter'),
      actions: [
        IconButton(
          tooltip: 'About UDFs',
          icon: const Icon(Icons.info_outline_rounded),
          onPressed: () => _showUdfInfoDialog(context),
        ),
      ],
    ),
    body: SafeArea(
      child: Center(
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 560),
          child: ListView(
            padding: const EdgeInsets.all(20),
            children: [
              _buildHeaderSection(context),
              const SizedBox(height: 24),
              _buildInputSection(context),
              const SizedBox(height: 16),
              _buildUnitSelectors(),
              const SizedBox(height: 20),
              _buildActionButtons(),
              const SizedBox(height: 24),
              if (_hasConverted && _convertedResult != null) _buildResultCard(context),
            ],
          ),
        ),
      ),
    ),
  );
}

Widget _buildHeaderSection(BuildContext context) {
  final colorScheme = Theme.of(context).colorScheme;
  return Column(
    children: [
      CircleAvatar(
        radius: 32,
```

```
        backgroundColor: colorScheme.primaryContainer,
        child: Icon(Icons.thermostat_rounded, size: 36, color: colorScheme.onPrimaryContainer),
      ),
      const SizedBox(height: 12),
      Text(
        'Convert between Celsius, Fahrenheit & Kelvin',
        textAlign: TextAlign.center,
        style: Theme.of(context).textTheme.titleMedium,
      ),
    ],
  );
}

Widget _buildInputSection(BuildContext context) {
  return TextField(
    controller: _inputController,
    keyboardType: const TextInputType.numberWithOptions(decimal: true, signed: true),
    textInputAction: TextInputAction.done,
    onSubmitted: (_) => _performConversion(),
    onChanged: (_) {
      if (_errorMessage != null) setState(() => _errorMessage = null);
    },
    decoration: InputDecoration(
      labelText: 'Enter temperature',
      hintText: 'e.g. 25 or 98.6',
      prefixIcon: const Icon(Icons.edit_rounded),
      suffixText: getUnitSymbol(_fromUnit),
      errorText: _errorMessage,
      errorMaxLines: 3,
    ),
  );
}

Widget _buildUnitSelectors() {
  return Row(
    crossAxisAlignment: CrossAxisAlignment.end,
    children: [
      Expanded(
        child: TemperatureDropdown(
          label: 'From',
          value: _fromUnit,
          onChanged: (unit) {
            if (unit == null) return;
            setState(() => _fromUnit = unit);
            if (_hasConverted) _performConversion();
          },
        ),
      ),
      Padding(
        padding: const EdgeInsets.symmetric(horizontal: 4),
        child: IconButton.filledTonal(
          tooltip: 'Swap units',
          icon: const Icon(Icons.swap_horiz_rounded),
          onPressed: _swapUnits,
        ),
      ),
      Expanded(
        child: TemperatureDropdown(
          label: 'To',
          value: _toUnit,
          onChanged: (unit) {
            if (unit == null) return;
            setState(() => _toUnit = unit);
            if (_hasConverted) _performConversion();
          },
        ),
      ),
    ],
  );
}

Widget _buildActionButtons() {
  return Row(
    children: [
      Expanded(
        child: OutlinedButton.icon(
          onPressed: _clearAll,
          icon: const Icon(Icons.clear_rounded),
          label: const Text('Clear'),
        ),
      ),
      const SizedBox(width: 12),
      Expanded(
        flex: 2,
        child: FilledButton.icon(
          onPressed: _performConversion,
          icon: const Icon(Icons.calculate_rounded),
          label: const Text('Convert'),
        ),
      ),
    ],
  );
}
```

```
  Widget _buildResultCard(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    final textTheme = Theme.of(context).textTheme;
    final fromSymbol = getUnitSymbol(_fromUnit);
    final toSymbol = getUnitSymbol(_toUnit);

    return FadeTransition(
      opacity: _fadeAnimation,
      child: ScaleTransition(
        scale: _scaleAnimation,
        child: Card(
          color: colorScheme.primaryContainer,
          elevation: 0,
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
          child: Padding(
            padding: const EdgeInsets.all(20),
            child: Column(
              children: [
                Text(
                  'Result',
                  style: textTheme.labelLarge?.copyWith(color: colorScheme.onPrimaryContainer),
                ),
                const SizedBox(height: 8),
                FittedBox(
                  fit: BoxFit.scaleDown,
                  child: Text(
                    '${_convertedResult!.toStringAsFixed(2)} $toSymbol',
                    style: textTheme.displaySmall?.copyWith(
                      fontWeight: FontWeight.bold,
                      color: colorScheme.onPrimaryContainer,
                    ),
                  ),
                ),
                const SizedBox(height: 8),
                FittedBox(
                  fit: BoxFit.scaleDown,
                  child: Text(
                    '${_inputTemperature!.toStringAsFixed(2)} $fromSymbol = '
                    '${_convertedResult!.toStringAsFixed(2)} $toSymbol',
                    style: textTheme.bodyLarge?.copyWith(color: colorScheme.onPrimaryContainer),
                  ),
                ),
                const Divider(height: 24),
                Text(
                  'Formula: ${getConversionFormula(_fromUnit, _toUnit)}',
                  textAlign: TextAlign.center,
                  style: textTheme.bodyMedium?.copyWith(color: colorScheme.onPrimaryContainer),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Listing 4.5: test/temperature_functions_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:temperature_converter/utils/temperature_functions.dart';

void main() {
  group('Conversion UDFs', () {
    test('1. celsiusToFahrenheit converts correctly', () {
      expect(celsiusToFahrenheit(0.0), closeTo(32.0, 0.001));
      expect(celsiusToFahrenheit(100.0), closeTo(212.0, 0.001));
      expect(celsiusToFahrenheit(25.0), closeTo(77.0, 0.001));
      expect(celsiusToFahrenheit(-40.0), closeTo(-40.0, 0.001));
    });

    test('2. fahrenheitToCelsius converts correctly', () {
      expect(fahrenheitToCelsius(32.0), closeTo(0.0, 0.001));
      expect(fahrenheitToCelsius(212.0), closeTo(100.0, 0.001));
      expect(fahrenheitToCelsius(98.6), closeTo(37.0, 0.001));
      expect(fahrenheitToCelsius(-40.0), closeTo(-40.0, 0.001));
    });

    test('3. celsiusToKelvin converts correctly', () {
      expect(celsiusToKelvin(0.0), closeTo(273.15, 0.001));
      expect(celsiusToKelvin(-273.15), closeTo(0.0, 0.001));
      expect(celsiusToKelvin(100.0), closeTo(373.15, 0.001));
    });

    test('4. kelvinToCelsius converts correctly', () {
      expect(kelvinToCelsius(273.15), closeTo(0.0, 0.001));
      expect(kelvinToCelsius(0.0), closeTo(-273.15, 0.001));
      expect(kelvinToCelsius(310.15), closeTo(37.0, 0.001));
    });

    test('5. fahrenheitToKelvin converts correctly', () {
      expect(fahrenheitToKelvin(32.0), closeTo(273.15, 0.001));
      expect(fahrenheitToKelvin(-459.67), closeTo(0.0, 0.001));
```

```
      expect(fahrenheitToKelvin(77.0), closeTo(298.15, 0.001));
    });

    test('6. kelvinToFahrenheit converts correctly', () {
      expect(kelvinToFahrenheit(273.15), closeTo(32.0, 0.001));
      expect(kelvinToFahrenheit(0.0), closeTo(-459.67, 0.001));
      expect(kelvinToFahrenheit(373.15), closeTo(212.0, 0.001));
    });
  });

  group('convertTemperature dispatcher', () {
    test('7. routes all six directions', () {
      expect(convertTemperature(100, unitCelsius, unitFahrenheit), closeTo(212, 0.001));
      expect(convertTemperature(100, unitCelsius, unitKelvin), closeTo(373.15, 0.001));
      expect(convertTemperature(212, unitFahrenheit, unitCelsius), closeTo(100, 0.001));
      expect(convertTemperature(212, unitFahrenheit, unitKelvin), closeTo(373.15, 0.001));
      expect(convertTemperature(373.15, unitKelvin, unitCelsius), closeTo(100, 0.001));
      expect(convertTemperature(373.15, unitKelvin, unitFahrenheit), closeTo(212, 0.001));
    });

    test('identity rule returns input unchanged', () {
      for (final unit in supportedUnits) {
        expect(convertTemperature(42.5, unit, unit), 42.5);
      }
    });
  });

  group('validatePhysicalTemperature', () {
    test('8. rejects values below absolute zero', () {
      expect(validatePhysicalTemperature(-1, unitKelvin), isNotNull);
      expect(validatePhysicalTemperature(-273.16, unitCelsius), isNotNull);
      expect(validatePhysicalTemperature(-459.68, unitFahrenheit), isNotNull);
    });

    test('accepts values at or above absolute zero', () {
      expect(validatePhysicalTemperature(0, unitKelvin), isNull);
      expect(validatePhysicalTemperature(-273.15, unitCelsius), isNull);
      expect(validatePhysicalTemperature(-459.67, unitFahrenheit), isNull);
      expect(validatePhysicalTemperature(25, unitCelsius), isNull);
    });
  });

  group('Presentation helpers', () {
    test('9. getConversionFormula returns the right formula', () {
      expect(getConversionFormula(unitCelsius, unitFahrenheit), '°F = (°C × 9/5) + 32');
      expect(getConversionFormula(unitKelvin, unitCelsius), '°C = K - 273.15');
      expect(getConversionFormula(unitCelsius, unitCelsius), contains('No conversion needed'));
    });

    test('10. getUnitSymbol returns the right symbol', () {
      expect(getUnitSymbol(unitCelsius), '°C');
      expect(getUnitSymbol(unitFahrenheit), '°F');
      expect(getUnitSymbol(unitKelvin), 'K');
      expect(getUnitSymbol('unknown'), '');
    });
  });
}
```

Listing 4.6: test/widget_test.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

import 'package:temperature_converter/main.dart';

void main() {
  testWidgets('converts 25 °C to 77 °F and clears', (WidgetTester tester) async {
    await tester.pumpWidget(const TemperatureConverterApp());

    await tester.enterText(find.byType(TextField), '25');
    await tester.tap(find.widgetWithText(FilledButton, 'Convert'));
    await tester.pumpAndSettle();

    expect(find.text('25.00 °C = 77.00 °F'), findsOneWidget);

    await tester.tap(find.widgetWithText(OutlinedButton, 'Clear'));
    await tester.pumpAndSettle();

    expect(find.text('25.00 °C = 77.00 °F'), findsNothing);
  });

  testWidgets('shows validation error for empty input', (WidgetTester tester) async {
    await tester.pumpWidget(const TemperatureConverterApp());

    await tester.tap(find.widgetWithText(FilledButton, 'Convert'));
    await tester.pumpAndSettle();

    // Inline error + SnackBar
    expect(find.text('Please enter a temperature value.'), findsNWidgets(2));
  });

  testWidgets('shows error for non-numeric input', (WidgetTester tester) async {
    await tester.pumpWidget(const TemperatureConverterApp());
```

```
    await tester.enterText(find.byType(TextField), 'abc');
    await tester.tap(find.widgetWithText(FilledButton, 'Convert'));
    await tester.pumpAndSettle();

    expect(find.text('Please enter a valid numeric value (e.g. 25 or 98.6).'), findsOneWidget);
  });

  testWidgets('rejects temperature below absolute zero', (WidgetTester tester) async {
    await tester.pumpWidget(const TemperatureConverterApp());

    await tester.enterText(find.byType(TextField), '-300');
    await tester.tap(find.widgetWithText(FilledButton, 'Convert'));
    await tester.pumpAndSettle();

    expect(find.textContaining('Absolute Zero'), findsOneWidget);
  });

  testWidgets('swap re-runs the conversion', (WidgetTester tester) async {
    await tester.pumpWidget(const TemperatureConverterApp());

    await tester.enterText(find.byType(TextField), '212');
    await tester.tap(find.widgetWithText(FilledButton, 'Convert'));
    await tester.pumpAndSettle();
    expect(find.text('212.00 °C = 413.60 °F'), findsOneWidget);

    await tester.tap(find.byTooltip('Swap units'));
    await tester.pumpAndSettle();
    expect(find.text('212.00 °F = 100.00 °C'), findsOneWidget);
  });
}
```

CHAPTER 5: Results

Output of the Temperature Converter

5.1 Examples of the application in action: The app opens with a text field, two unit dropdowns with a swap button, and Clear and Convert buttons. After a valid conversion a result card shows the converted value, the full statement such as 25.00 °C = 77.00 °F and the formula used. Figures 5.1 and 5.2 show the screen on a phone, Table 5.1 lists benchmark temperatures calculated with the six formulas, and Table 5.2 lists the behaviour of the screen for typical inputs as checked by the widget tests.

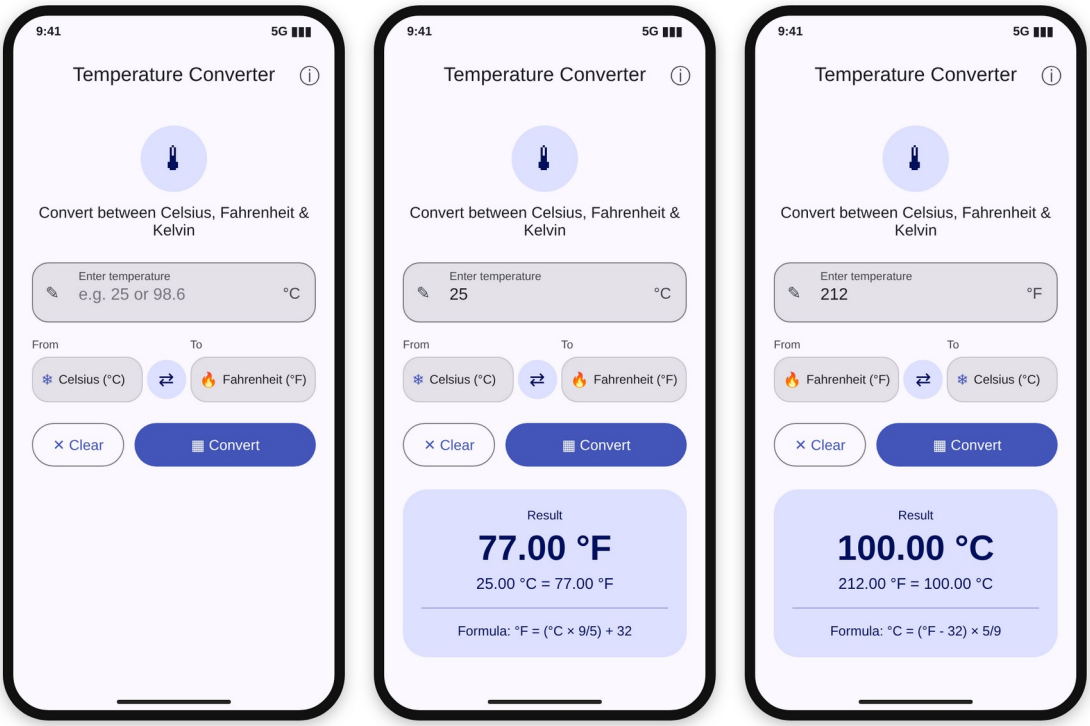


Figure 5.1: Initial screen, 25 °C to °F result, and 212 °F to °C result after swapping units

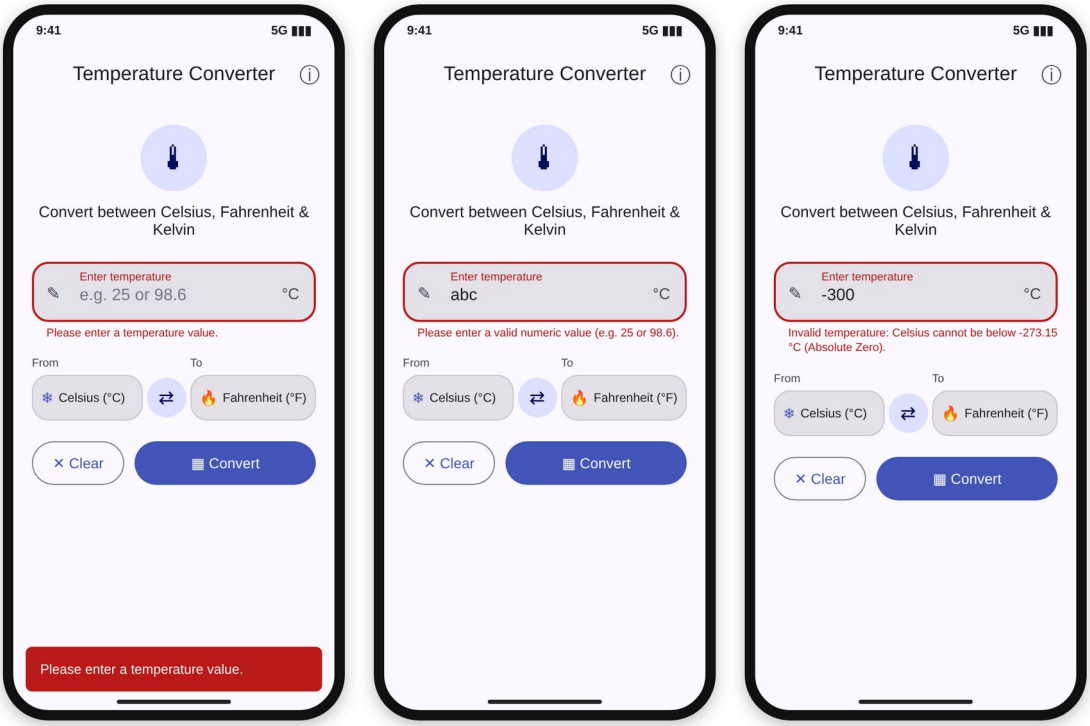


Figure 5.2: Validation messages for empty input, non-numeric input and a value below absolute zero

Table 5.1: Benchmark conversions

Physical point	Celsius	Fahrenheit	Kelvin
Absolute zero	-273.15	-459.67	0.00
Cross-over point	-40.00	-40.00	233.15
Freezing point of water	0.00	32.00	273.15
Room temperature	25.00	77.00	298.15
Human body temperature	37.00	98.60	310.15
Boiling point of water	100.00	212.00	373.15

Table 5.2: Behaviour of the screen

Input and units	Response of the application
25 in Celsius to Fahrenheit	Result card: 25.00 °C = 77.00 °F
212 in Celsius to Fahrenheit	Result card: 212.00 °C = 413.60 °F
Swap after the previous result	Units exchange and the card shows 212.00 °F = 100.00 °C
Empty field	Error: Please enter a temperature value. (inline and SnackBar)
abc	Error: Please enter a valid numeric value (e.g. 25 or 98.6).
-300 in Celsius	Error: Invalid temperature: Celsius cannot be below -273.15 °C (Absolute Zero).
Clear button	Field emptied, units reset to Celsius and Fahrenheit, result removed

5.2 Discussion of the results: The project contains 12 unit tests for the UDFs and 5 widget tests for the screen. The unit tests use closeTo(expected, 0.001) instead of exact

equality because floating point numbers can carry tiny rounding errors. They cover all six conversions, the dispatcher in every direction, the identity rule when both units are the same, the absolute zero checks and the helper functions. The widget tests simulate typing and tapping to check the result text, the three error messages, the swap and the clear actions. The tests are run with flutter test and the code is checked with flutter analyze.

CHAPTER 6: Conclusion and Future Scope

Summary of the project

6.1 Recap of the project's objectives and achievements: The project set out to demonstrate User Defined Functions in a real Flutter application. It separates the calculation logic from the interface, implements ten pure UDFs, validates input defensively including the absolute zero limit, offers a polished Material 3 interface and is backed by automated unit and widget tests.

6.2 Discussion of the project's limitations: The converter supports only three temperature scales and converts one value at a time. It does not keep a history of conversions and the result is displayed with two decimal places. The app works offline and does not read the current temperature from a device or a weather service.

Future scope and enhancements

6.3 Ideas for future improvements and enhancements: More scales such as Rankine and Delisle can be added with new UDFs. A history of recent conversions can be stored with shared_preferences or SQLite. Voice input, a live weather service for the current city and conversion of other units such as length and mass would turn the project into a general unit converter.

References

[1] Flutter documentation, https://docs.flutter.dev

[2] Dart language tour, https://dart.dev/language

[3] Effective Dart guidelines, https://dart.dev/effective-dart

[4] Flutter testing overview, https://docs.flutter.dev/testing/overview

[5] Material Design 3, https://m3.material.io

[6] R. C. Martin, Clean Architecture, Prentice Hall, 2017.

[7] Bureau International des Poids et Mesures, The International System of Units (SI), 9th Edition, 2019.